Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA [B.Tech - ECE, EEE, CSE, IT, MECH, Civil & Chemical]
Sangivalasa 531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712009222 Website: www.anits.edu.in email

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026	Semester:	1st Sem 3rd Yr
Name of the Student:	Sk. Shabaz	Roll Number:	322126552275
Name of the Lab:	Full Stack	Timings:	9:40 - 12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva- Voce (2Marks)			
1/7	1					11		
8/7	2					11		
15/7	3					12		
22/7	4					11		
29/7	5					12		
5/8	6					11		
	7							
	8							
	9							
	10							

Name of the Faculty:

Faculty Signature:

1/7/26

Introduction to HTML and CSS

→ HTML :- Hyper Text markup Language is the standard markup language used to create and Structure Content on web page. It defines the basic structure of webpage using Elements and tags such as headings, paragraphs, links, Images, lists, tables, forms and other multimedia Content. HTML documents are interpreted by web browsers to display the Content in a Structured and meaningful manner. HTML provides the Structure and Semantic organization of a webpage.

→ CSS :- Cascading StyleSheet is the Stylesheet language used to Control the presentation and visual Appearance of HTML documents. It is used to define properties such as Colors, Fonts, backgrounds, Spacing, borders, Alignment dimensions and layouts. CSS also supports Responsive design Allowing webpages to Adapt to different screen size and devices. CSS can be Applied using Inline, Internal or External Style Sheet.

→ HTML & CSS work together to develop webpage, where HTML provides the Structure and Content. While CSS provides the Styling layout, Visual Representation. The Combination of both forms the foundation of modern web development and Enables the creation of organized Attractive and user-friendly web Interfaces.

Scanned with OKEN Scanner

Difference between MERN Stack and Full Stack

→ MERN Stack is a Javascript based Technology stack used for developing Modern web application MERN stands for MongoDB, ExpressJs, ReactJs, NodeJs, MongoDB is used for database Managment, ExpressJs handles the backend framework and Api's, ReactJs is used to develop frontend user interface and NodeJs provides the server side Runtime Environment.

→ Full-Stack development refers to the development of both the frontend and backend of a Web Application Along with database Integration. A Full stack developer works with Technologies used for designing the User Interface Implementing server-side logic Managing databases.

HTML Components:-

1. HTML Tags - Define the Structure and Elements of webpage
2. Heading - used to display titles and headings using <h1> to <h6>
3. paragraph - used to display text using the <p> tag.
4. Links - used to Navigate between webpage using <a> tag.
5. Images - used to display Images using <image> tag.
6. lists - used to create ordered and unordered lists using <OL>, <UL>
7. Table - used to organize data in rows and columns
8. Forms - used to collect user Input using elements such as text Boxes buttons and Checkboxes.
9. Div and Span - used to Group and organize webpage content
10. Semantic Elements - Elements such as <header> <nav>, <section>, <article>, <footer> provides Meaningful webpages Structure.

Scanned with OKEN Scanner

CSS Components :-

→ Selectors - Select HTML Elements to apply Styles
→ properties - Define the visual characteristics of Elements
→ Values - Specify the setting of a css property
→ Colors - Define text, background, border colors.
→ Fonts - Control font family, size style and weight
→ Backgrounds - Set Background colors and images
→ Borders - Define the border style, width and radius.
→ Margin - Controls the space outside Element outside an Ele
→ padding - Controls the space inside Element
→ layout - Controls the Managment of Elements using checkbox, grid & positioning.

HTML vs HTML5

HTML	HTML5
Older version of HTML	Latest major version of HTML
Limited multimedia support	support audio and video
uses External plugins for multimedia	No External plugs needed
Has fewer semantic tags	provides semantic Tags
No <canvas> Element	Support <canvas>
Limited local Storage	Support local Storage
Complex DOCTYPE declaration	Simple

Scanned with OKEN Scanner

CSS Vs CSS 3

CSS	CSS 3
Earlier Version of CSS	Advanced Version of CSS
Basic Styling features	provides advanced Styling features
limited Animation	Supports Animations and Tranition.
No flexible layout system	Support flexble and Grid
limited Visual Effects	Supports Shadows, Gradients Effects.
No media queries	Support media queries
Uses a single specification	Divide into different modules.

Scanned with OKEN Scanner

2 Design a static webpage using HTML and CSS pomodoro timer using HTML, CSS

```
<!DOCTYPE html>
<html>
  <head>
    <title> pomodoro Timer </title>
    <style>
      body {
        font-family : Arial ;
        background: #f5f5f5;
        text-align :center;
        padding : 50px;
      }
      .timer {
        width: 300px;
        margin: auto;
        padding : 30px;
        background : white;
        border-radius : 15px;
        box-shadow :00 10px gray;
      }
      h1{
        color: #e74c3c;
      }
      .time {
        font-size: 50px;
        margin: 20px;
      }
      button{
        padding : 10px 20px;
        margin: 5px;
        border: none;
        border-radius : 5px;
        background :#e74c3c;
        color :white ;
      }
    </style>
  </head>
  <body>
    <div class ="timer">
      <h1> pomodoro Timer </h1>
      <div> class = "time"> 25:00 </div>
      <button> Start </button>
      <button> reset < /button>
    </div>
  </body>
</HTML>
```

Design an HTML code using audio Tags

```
<!DOCType html>
<Html>
  <heads>
    <title >Audio Example </title>
  <lhead>
 <body>
     <h2> MY Favorite Song </h2>
      <audio Controls>
         <source src = " Song.mp3" type ="audio impeas">
           your browser doesn't Support the Audio tag
         <laudio>
 <lbody>
<!HTML>
```

O/p:- Display an audio player with play, pause and volume Controls

Steps:-

Create a New HTML file

Add the basic HTML Structure

Add heading using <h2> Tag

Omit the <audio> Tag with Controls Attribute

Specify the audio file using <Sources> tag

Save and open the HTML file in a Web browser

play the Audio using displayed Controls.

Scanned with OKEN Scanner

ii Design a HTML Code using video Tags

```
<!DOCType HTML>
<html>
<head>
 <title> Video Example </title>
</head>
<body>
 <h2> My video </h2>
 <video width = "400" Controls>
  <source src = "video.mp4" type = "video/mp4">
 </video>
</body>
</html>
```

Q)p:- Design a video player with play, pause, Volume and Full screen Controls

Steps:-

Create a New HTML File

Add a basic HTML Structure.

Add a Heading using the <h2> tag

Insert the <video> tag with Controls attribute

Specify the video file using <Source> tag

Save the file & open it in web browser

play the video using displayed Controls.

Scanned with OKEN Scanner

3. Write a program using Javascript (arrays, functions)

Code:-

```
let Numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11];

function display Array (arr){
        Console.log ("Array Elements:");
    for (let i=0 ; i< arr.length ; i++){
            Console.log (arr[i]);
    }
}

function find sum (arr){
    let sum=0;
    for (let i=0; i<arr.length ; i++){
        sum+=arr[i];
    }
    return sum;
}
```

O/P:- Array Element:-

12
45
8
23
56
19
34
67
90
11

Sum = 865.

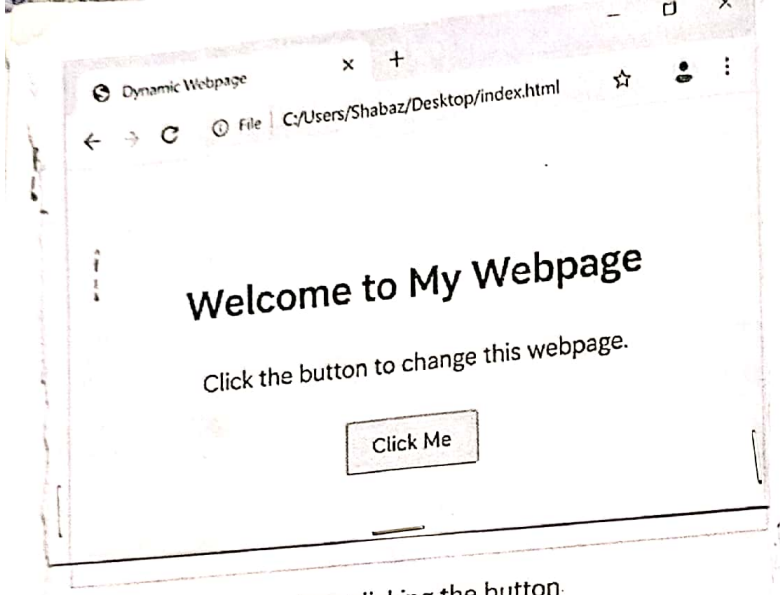
Experiment-4

Aim: To Develop Dynamic webpage using Javascript

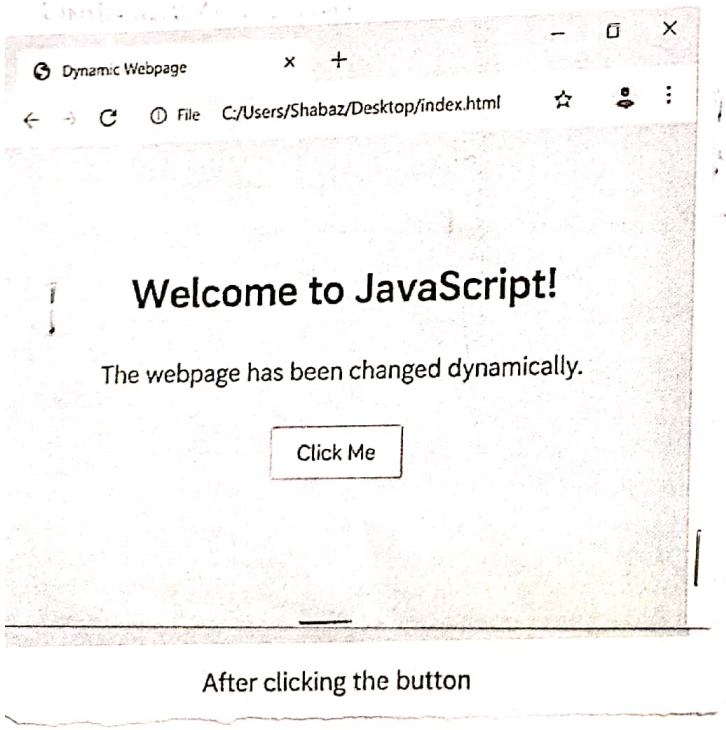
Source Code:-

```
<!Doctype html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
          initial-scale=1.0">
 <title> Dynamic webpage </title>
 <style>
   body {
      font-family: Arial, sans-serif;
     text-align: center;
     margin: 0;
     background-color: #f2f2f2;
    }
   header {
      background-color: #333;
      color: white;
      padding: 20px;
    }
  .container {
     margin: 40px auto;
     padding: 25px;
     background: white;
     width: 80%;
     max-width: 400px;
     border-radius: 10px;
     box-shadow: 0 0 10px gray;
   }
  input {
     padding: 12px;
     width: 85%;
     margin: 10px 0;
     border: 1px solid #ccc;
     border-radius: 15px;
   }
  button {
      padding: 10px 20px;
      Margin: 10px 5px;
      border: None;
      border-radius: 5px;
      background-color: #333;
      Color: white;
      cursor: pointer;
   }
   button:hover {
       background-color: #555;
    }
  #Menage {
       margin-top: 20px;
       font-size: 18px;
```

Scanned with OKEN Scanner



Before clicking the button



After clicking the button

Experiment-5

Implement Modules in Node JS

Aim:- To Implement Modules in Node JS.

Source code:-

Modules:-

```
project /
  ├add.js
  ├sub.js
  ├mul.js
  ├ div.js
  └ app.js
```

→ add.js:-

```
Function add (a,b){
    return a+b;
}
Module.Exports = add;
```

→ sub.js:-

```
Function sub (a,b) {
    return a-b;
}
Module.Exports = sub;
```

→ mul.js:-

```
Function mul(a,b) {
    return a*b;
}
Module.Exports = mul;
```

Scanned with OKEN Scanner

→ div.js

```
Function div (a,b) {
        return a/b;
}
Module.Exports = div;
```

→ App.js

```
Const add = require ("./add");
Const sub = require ("./sub");
Const mul = require ("./mul");
Const div = require ("./div");

let a = 20;
let b = 10;

Console.log ("Addition: ", add(a,b));
Console.log ("Subtraction: ", sub(a,b));
Console.log ("multiplication:", mul(a,b));
Console.log ("division :", div(a,b));
```

Bash:- Node App.js

Output :

Addition: 30
Subtraction: 10
multiplication 200

Experiment 6

→ develop mini Application using Express and Node.js

Aim:- To develop mini Application using Express and Node.js

Source code Process

Install Node.js on your Computer, After Installation

Check it with:-

Git bash

```
Node -- version
Npm --version
```

Install Express

```
npm install Express
```

Structure

```
Express-mini-app /
        └ Express.js
        └ Javascript /
                └ Index.HTML
```

The Folder Name Javascript must match the Name and Useit

JS

```
App.use (Express.static ("Javascript"));
```

Scanned with OKEN Scanner

Express Js

```
Const Express = require ('express');
Const app = Express();
Const port = 3000;
//middle ware to read JSON data
app.use (Express.Json());
app.use (Express.static('Javascript'));
//login Api
api.post ('/login', (req, res) => {
    Const {Email, password} = req.body;
    If (Email === 'admin@gmail.com' && password === '12345') {
        res.Json({
            success: True}
            menege: 'login Succesful'
        });
    }else{
    res.Json({
        success: false
        menege: 'Invalid Email or password !!'
        });
    }
});
//About route
App.get ('/about', (req, res) => {
});
//Start server
app.listen (port, () => {
    Console.log ('Server is running on port 3000');
    Console.log (' visit http://local host :${port}');
});
```

Java Script Understand time

```
<!Doctype HTML>
<html>
<head>
 <title> Express Mini Application </title>
 <style>
   body {
      font-family: Arial;
      text-align: Center;
      margin-top: 80px;
      background-color: #f2f2f2;
   }
  .Container {
      back ground :white;
       width: 350px;
       margin: auto;
       padding: 30px;
      border-radius: 10px;
      box-Shadow: 0 0 10 px gray;
   }
```

Scanned with OKEN Scanner

```
input{
    width: 90%;
    padding: 10px;
    margin: 10px;
}
button{
    padding: 10px 20px;
    margin: 10px;
    cursor: pointer;
}
#result{
    margin-top: 20px;
    font-weight: bold;
}
</style>
</head>
<body>
<div class="container">
    <h2> login </h2>
<input type="email" id="email" placeholder="Enter Email">
<input type="password" id="password" placeholder="Enter password">
<br>
<button onclick="login()"> login </button>
<button onclick="resetForm()"> Reset </button>
<p id="result"></p>
<a href="/about"> about us </a>
</div>
<script>
function login() {
    const email = document.getElementById("email").value;
    const password = document.getElementById("password").value;

fetch('/login', {
    method: 'post',
    headers: {
        'Content-Type': 'application/json'
    },
body: JSON.stringify({
        email: email,
        password: password
    })
})
.then(response => response.json())
.then(data => {
    document.getElementById("result").innerHTML =
        data.message;
});
}
function resetForm(){
    document.getElementById("email").value;
    document.getElementById("password").value;
    document.getElementById("result").value;
```

</script>
</body>
</html>

Commands to Execute

cd . Expru - mini - app

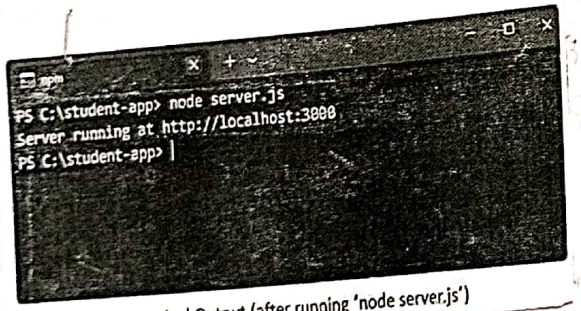
Install Expru

Npm Install Expru

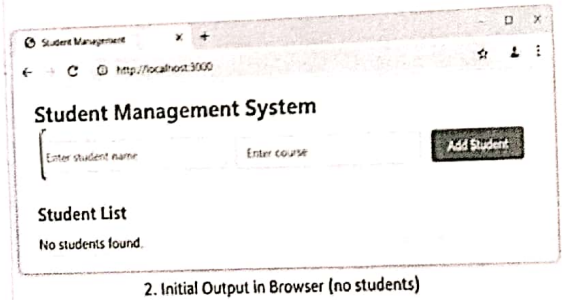
Node Expru.js // Start Server

Server is Running on port 3000

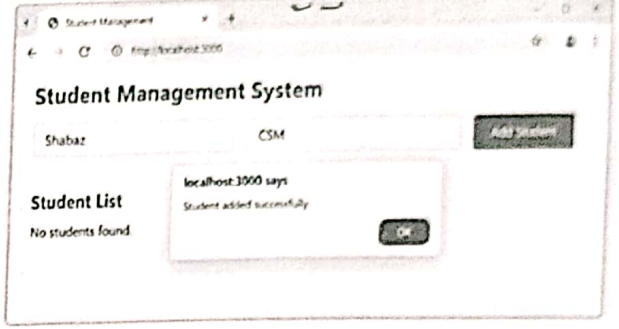
Visit http://local host:3000



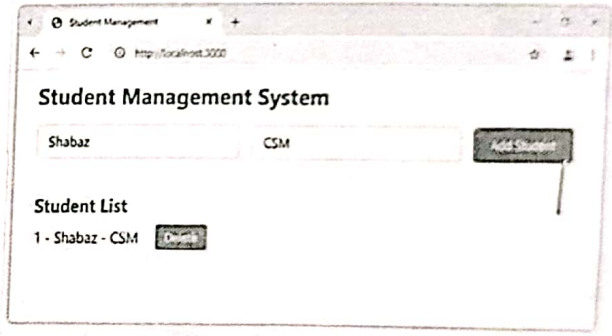
1. Terminal Output (after running 'node server.js')



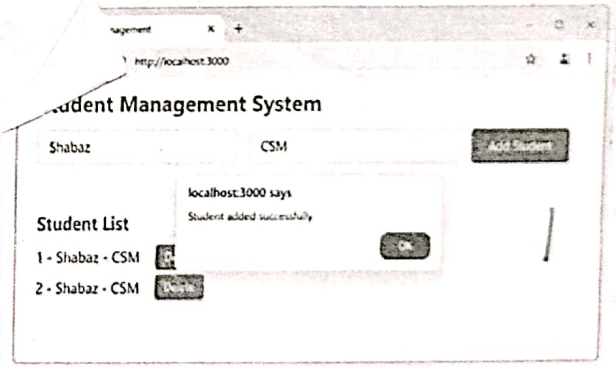
2. Initial Output in Browser (no students)



3. After adding a student (alert message)

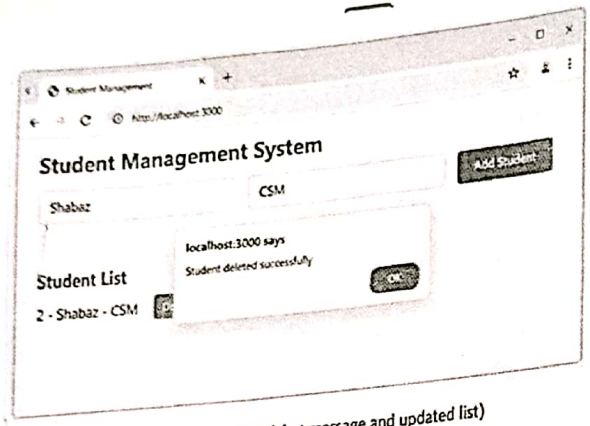


4. Student list after adding one student



5. After adding another student (alert message)

Scanned with OKEN Scanner



6. After deleting a student (alert message and updated list)

Experiment -1

Create a functional Component and class Component using react.

Aim :- To develop a react Application using Functional Components using Functional Components and class Compon... to display Student details, Manage state, Handle button click and dynamically Update data on a Web page.

Functional Component :-

A Functional Component is a Javascript function that Return JSX to display the User Interface. It uses React hooks such as useState to manage State and Update the Webpage dynamically.

Program :-

```
Import React, {useState} from
'React':
function StudentFunctional() {
        Const [name, SetName] = useState ["Shabaz"
        Const [Course, setCourse] = UseState ("CSM"
        Const [count, SetCount] = useState (0);
        Const changeName = () => {
                setName ("Shaik");
        };
```

Scanned with OKEN Scanner

```
Const changeCourse = () => {
    setCourse("CSD");
};
Const incrementCount = () => {
    setCount(Count + 1);
};

Return (
  <div style={{
      border: "2px solid blue",
      padding: "20px",
      margin: "20px",
      borderRadius: "10px",
      textAlign: "center"
  }}>
  <h1> Student Details </h1>
  <h2> Functional Component </h2>
  <p> <b> Name: </b> {name} </p>
  <p> <b> Course: </b> {course} </p>
  <p> <b> Button Click: </b> {count} </p>
  <button onClick = {changeName}>
      Change Name </button>
  <button onClick = {changeCourse}> Change
```

```
  <button onClick = {incrementCount}> Increment
      </button>
  </div>
);
```

Output:- When the user clicks the button, the user name, course & count change dynamically without reloading the webpage.

Class Component

A class component is an ES6 JavaScript class that extends React.Component. It uses the render() method to return JSX and manages state using this.state and this.setState().

Program:-

```
import React, {Component} from
  'react';
class Student class extends Component {
    Constructor (props ){
        super (props);
        this.state = {
          name = "Shaba2",
          Course : "Computer Science",
          Count : 0
        };
    }
  ChangeName = () => {
        this.setState ({
          name: "Shaik"
        });
  };
  ChangeCourse = () => {
        this.setState ({
          Course : "CSM"
        });
  };
Increment Count = () => {
    this.setState ((prevState) => {
          Count = prevState.Count + 1
    }));
  };
Render () {
  Return (
    <div Style = {{
          border = "2px Solid green";
          padding = "20px";
          Margin = "20px";
          borderRadius = "10px";
          textAlign = "Center"
    }}>
    <h1> Student Details </h1>
    <h2> Class Component </h2>
<p><b> Name: </b> {this.state = name} </p>
<p> <b> Course : </b> {this.State Course} </p>
<p> <b> Button Click : </b> {this.state.Count
                                    </p>
<button OnClick = {this.ChangeName}> ChangeName
  </button>
```

Scanned with OKEN Scanner

<button onClick = {this.changeCourse}>
Change Course </button>

<button onClick = {this.incrementCount}>
Increment </button>

Experiment -8

Implement http methods using JSON server

Aim:- To Implement HTTP methods such as GET, post, put, PATCH and DELETE using JSON server to perform CRUD (Create, Read, Update & delete) Operation on Student records.

Theory:- JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server. from Search.

HTTP Methods are used to Communicate btwn a Client and a server.

- GET - Retrieves data from the server
- POST - Adds New data to the server
- PUT - Replace an Existing record
- PATCH - Update selected fields of a record
- DELETE - Remove a record from the server

a) Create a JSON server:

Step 1: Initialize the project

Open the vs Code terminal & execute the following commds

```
mkdir JSON server
cd JSON server
npm init -y
npm install JSON server
```

Step2:- Create db.json

Create a file Named db.json and add the following student data

```
{
  "Students" = {
      {
       "id": "1",
       "name": "Shabaz",
       "Course": "CSM",
       "Marks": 90;
      };
    {
     "id": "2",
     "name": "Shaik",
     "Course": "CSM",
     "Marks": 90
    }
    {
     "id" : "3"
     "name": "Hemanth",
     "Course": "IOT",
     "Marks": 90
    }
```

Step3:- Start the JSON Server

for the current JSON Server Version, Execute

npx .JSON. Server db.JSON

The Server start at

http://localhost:3000

The Student records can be Account using

http://local host : 3000/ Students.

2-Implementation of HTTP Methods

a) GET Method:- The GET Method is used to retrieve Student record from JSON Server

Code:

```
Fetch ('http://local host :3000/ Students')
    .then (response => response.JSON ())
    .then (data => {
          Console .log(data);
      })
    .catch (Error => {
          Console log (error);
      })
```

Scanned with OKEN Scanner

B. **POST Method** :- the POST method is used to insert a new student record into the server

Code :-
```
fetch ('http://localhost:3000/students', {
    method: 'POST',
    headers: {
        'Content-type': 'application/JSON'
    },
    body: JSON.stringify ({
        name: 'Rahul',
        Course: 'CSE',
        Marks: 92
    })
})
.then (response => response.JSON())
.then (data => {
    Console.log (data);
})
.catch (error => {
    Console.log (Error);
});
```

o/p:-

c) **PUT Method** : The PUT Method is used to replace an existing student record with updated data.

Code:-
```
Fetch ('http://localhost:3000/students', {
    method: 'put',
    headers: {
        'Content-Type': 'Application/JSON'
    },
    body: JSON.stringify ({
        name: "Teja Narayan",
        Course: "AIML",
        Marks: "95"
    })
})
.then (response => response.JSON())
.then (data => {
    Console.log (data); })
```

Scanned with OKEN Scanner

```
.catch (Error => {
    Console.log (Error);
});
```

O/p

D) PATCH METHOD:- The PATCH method is used to Update only selected fields of an existing student record

Code:-
```
fetch ('http://localhost:3000/students/2', {
    method: 'PATCH',
    header: {
      'Content-Type': 'Application/JSON'
    },
    body: JSON.stringify {
    mark1: 96
    }
  })
.then (response => response.JSON())
.then (data => {
    Console.log (data);
  })
.catch (Error => {
    Console.log (Error);
  });
```

O/p:-

E) DELETE Method :- The DELETE method is used to remove an existing student record from the server.

Code:-
```
Fetch ('http://localhost:3000/students/3', {
    method: 'DELETE'
  })
.then (response => {
    if (response.OK) {
      Console.log ("Student deleted successfully");
    }
  })
.catch (Error => {
    Console.log (Error)
  });
```

Scanned with OKEN Scanner

```
// Read :- Get Student by id
app.get('/student/:id', async (req, res) => {
    try {
        const student = await
        Student.findById(req.params.id);
        if (!student) {
            return res.status(404).json({
                message: "Student not found"
            });
        }
        res.status(200).json(student)
    } catch (error) {
        res.status(400).json({message:
            Error message});
    }
});

// Update
app.put('/student/:id', async (req, res) => {
    try {
        const student = await
        Student.findByIdAndUpdate(
            req.param.id,
            req.body,
            {new: true, runValidation: true}
        );
        if (!student) {
            return res.status(404).json({
                message: "Student not found"
            });
        }
        res.status(200).json(student);
    } catch (error) {
        res.status(400).json({message: error.
            message});
    }
});

// DELETE
app.delete('/students/:id', async (res.req.assign) {
    try {
        const student = await Student
            findById AND Delete(
            req.params.id
            );
        if (!student) {
            return res.status(404).json({
            message: "Student not found"
            });
        }
```

```
Const PORT = 3000;
app.use (express.json());
mongoose.Connect ('mongodb://127.0.0.1:
                    27017/College)
.then (()=> Console.log ("MongoDB Connectivity")
.catch (Err=>Console.log (Err));

// create Student.
Const Student Schema = new
  mongoose.Schema ({
      name: {
        type: String,
        required: true
      },
      Course: {
        type: String
        required: true
      },
      mark: {
        type: Number
        required: true
      }
  });
```

```
// create Student model
  Const Student =
      mongoose.model ('Student', StudentSchema)

//create :API
  app.post ('/students', async (req, res)=> {
      try {
       Const Student = new Student (req.body);
       Const saved Student = await Student.saved;

  res.status (201).JSON (Saved Student);
  } catch (Error) {
       res.status (400).JSON ({message:
                                Error: message});
      }
  });

//READ:-Get all Students
  app.get ('/students', async (req, res)=> {
       try {
       Const Students: await
       Student.Find ();
            res.status (500).JSON ({message: Error
                                    message});
       }
  });
```

Scanned with OKEN Scanner

O/p

•After Executing the POST, PUT, PATCH & DELETE Operations in sequance the remaining student will be

O/p:

Thus,

Thus, HTTP methods GET, POST, PUT, PATCH and delete were succesfully implemented using JSON server the CRUD operations were performed on Student records. The data was retrieved, inserted, updated & deleted using HTTP requests.

Experiment -9

Develop CRUD operations using MongoDB

Aim:- to develop a web Applications using Node.JS, Express.JS and MongoDB to perform CRUD operations (Create, Read, Update, delete) on student Records.

Theory:-

MongoDB is a NoSQL database that stores data in flexible, JSON like document CRUD operations are used to manage data in a database

Mongoose is an object data Modeling (ODM) library for MongoDB and Node JS. It helps define Schemas, interact with Collections and perform database operations.

Installation and project setup:-

```
mkdir MongoDB-CRUD
cd MongoDB-CRUD
npm init -y
npm install express mongoose
```

program:- Server.JS

```
Const Express = require("Express");
Const mongoose = require("mongoose");
Const app = Express();
Const PORT = 6000;
app.use(Express.json());
mongoose.connect("mongodb://127.0.0.1:27017/College
.then(() => console.log("MongoDB Connected"))
```

Scanned with OKEN Scanner

```
.catch (err => Console.log(Error));

//Create Student
Const Student schema = new
mongoose.Schema({
    name: {
      type: String;
      required: true
    };
  Course : {
    type: String
    required: true
  },
  marks: {
    type : Number
    required: true
  }
});
// Create Student model
Const Student =
      mongoose.model('Student', Student schema)
//Create : Add
app.post('/Students', async (req, res) => {
    try {
    Const Student = new Student (req.body);
    Const saved Student = await student.save();
    res. status (201) .json (saved Student);
} Catch (Error) {
    res. status (400) .json ({message: Error
                              :message});
  }
});
// Read : GET all Student
app.get ('/Students', async (req, res) => {
    try {
    Const Student : await
    Student .Find();
    . res. statUp (500). Json ([message-Error.
                                message});
  }
});
// Read Get Student by ID
app.get ('/Student s /id', async (req, res) => {
    try {
    Const Student : await
    Student Find By Id (req. params. id);
    if (! Student) {
        return res. status (401). Jso n ({
          message. "Student not found"
        });
      }
    res. status (200) .Json (Student)
    } catch (Error) {
```

Scanned with OKEN Scanner

```
    res.status(400).json({message: error.message})
  }
});
// Update
app.put('/students/:id', async (req, res) => {
  try {
    const student = await
    Student.findByIdAndUpdate(
      req.params.id,
      req.body,
      {new: true, runValidators: true}
    );
    if(!student){
      return res.status(404).json({
        message: "Student not found"
      });
    }
    res.status(200).json(student);
  } catch (error) {
    res.status(400).json({message: error.message})
  }
});
```

```
// DELETE
app.delete('/students/:id', async (req, res) => {
  try {
    const student = await Student
      .findByIdAndDelete(
        req.params.id
      );
    if(!student){
      return res.status(404).json({
        message: "Student not found"
      });
    }
    res.status(200).json({
      message: "Student deleted successfully"
    });
  } catch (error) {
    res.status(400).json({message:
      error.message});
  }
});
```

Scanned with OKEN Scanner